EraseGraph: Verifiable Deletion Propagation in Distributed Systems

Haoyuan Shan & Vritika Malhotra
Master's in Software Engineering
San Jose State University
San Jose, United States
haoyuan.shan@sjsu.edu
vritika.malhotra@sjsu.edu

Sakshat Patil & Asim Mohammed
Master's in Software Engineering
San Jose State University
San Jose, United States
sakshatnandkumar.patil@sjsu.edu
asim.mohammed@sjsu.edu

***Abstract*—Modern applications store user data in multiple places at the same time, such as the main database, cache, search index, analysis storage and backup system. Therefore, a simple deletion operation in a database is not enough to support privacy requirements. In this project, we built EraseGraph, a distributed deletion system that can send a deletion request to multiple services, track step-level progress, record proof events, and display the results in the network dashboard. Our design uses event-driven workflows, including RabbitMQ, NestJS service, PostgreSQL, Redis and React front-end. We have also added retry queues, dead letter queues, repeat event protection, circuit breaker logic, encryption proof, and observability through Jaeger, Prometheus and Grafana. The system is deployed locally in Docker Compose and Google Kubernetes Engine for cloud demonstration. The final result is an end-to-end platform that can show the audit readiness of successful deletion, temporary failure and recovery, partial completion, and what happened in each service. This report summarizes the problems, design, implementation, deployment, testing and lessons learned.**

***Index Terms*—distributed systems, deletion workflow, audit trail, event-driven architecture, RabbitMQ, Kubernetes**

I. Introduction

In real software systems, user data is usually copied into many components. A user profile may exist in a main relational database, Redis cache, search index, analytics tables, and backup metadata. If only one copy is removed, the system cannot honestly say the user data has been deleted everywhere.

The main goal of EraseGraph is to make deletion propagation visible, reliable, and verifiable. Instead of treating deletion as a single database call, we model it as a distributed workflow. A request is created once, then multiple cleanup services process it, each reporting its result back to a central coordinator. The system keeps a request state, step states, proof events, and a frontend view for operators.

Our scope includes orchestration, retry logic, proof generation, bulk upload, admin monitoring, and cloud deployment. We do not claim to solve all real compliance problems. For example, we do not implement full enterprise identity resolution, legal retention rules, or hardware-backed key management. Our goal is a clear and realistic educational system that demonstrates important distributed systems ideas.

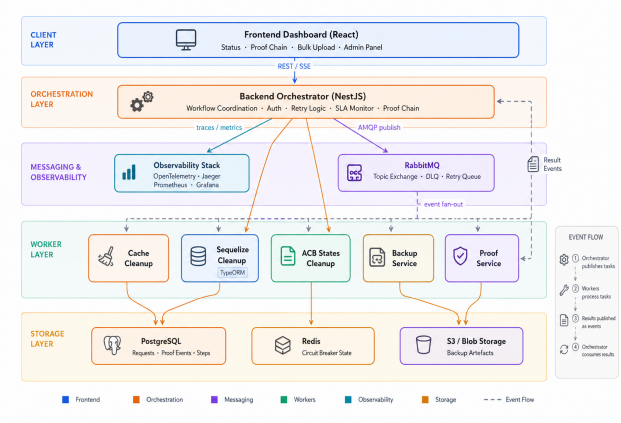

Fig. 1. EraseGraph high-level architecture.

II. Requirements and Use Cases

We designed EraseGraph around a simple but realistic user story: an operator submits a deletion request for one subject ID, and the system must try to remove that user's data across several downstream systems.

A. Functional Requirements

The main functional requirements were:

1) Accept one deletion request and generate a request ID and trace ID.
2) Propagate deletion work to multiple services.
3) Track overall request status and step-level status.
4) Record proof events for audit and later verification.
5) Support retry and failure recovery for temporary service outages.
6) Show request history, proof timeline, demo users, bulk upload, and admin monitoring in the frontend.
7) Support both local and cloud deployment.

We also added a batch CSV deletion feature so an operator can upload multiple subject IDs in one request. This made the project more useful for an admin-style workflow instead of only a single-user demo.

B. Non-Functional Requirements

The main non-functional requirements were reliability, observability, maintainability, and demo readiness. Reliability means the system should not lose deletion work on the first failure. Observability means operators should be able to inspect traces, queue behavior, health state, and stalled requests. Maintainability means each service should have a clear responsibility.

C. Use Cases

The main use cases are:

- A privacy operator deletes one user and tracks the result.
- A reviewer checks proof events to understand which service succeeded or failed.
- An admin sees that one service is unhealthy or that a request has been stuck too long.
- A team demonstrates recovery from temporary failure using retry queues and DLQ behavior.

III. System Design

EraseGraph uses a distributed microservice design. The main components are a React frontend, a backend orchestrator, several cleanup services, a proof service, RabbitMQ, PostgreSQL, Redis, and an observability stack.

A. Architecture

The backend receives `POST /deletions`, stores a new request in PostgreSQL, creates deletion steps, and publishes a `DeletionRequested` event. Cleanup services subscribe to the message broker and process different parts of the workflow. For example, one service removes primary data, another clears cache, another updates search, another marks backup cleanup, and another simulates delayed analytics cleanup. Each service publishes step-result events such as success, retrying, or failure. The backend consumes these result events and updates the main request state.

RabbitMQ is used because it fits asynchronous fan-out well and lets us model retries and dead-letter behavior clearly [1]. Redis is used for fast state such as circuit breaker status. PostgreSQL stores requests, steps, users, processed events, and proof events.

B. State Model

Each deletion request follows a lifecycle such as *PENDING*, *RUNNING*, *COMPLETED*, *PARTIAL_COMPLETED*, *FAILED*, or *RETRYING*. Step-level states give more detail. This is important because distributed deletion does not finish everywhere at the same time. For example, the analysis and cleaning is deliberately delayed to show the final consistency.

PARTIAL_COMPLETED is an intentional state in our project. This means that the request has progressed, but not every downstream step has been completely successful. In one case, it appears temporarily, and the analysis is still catching up. In another case, if the circuit breaker skips a service and the other service succeeds, it can become the final state.

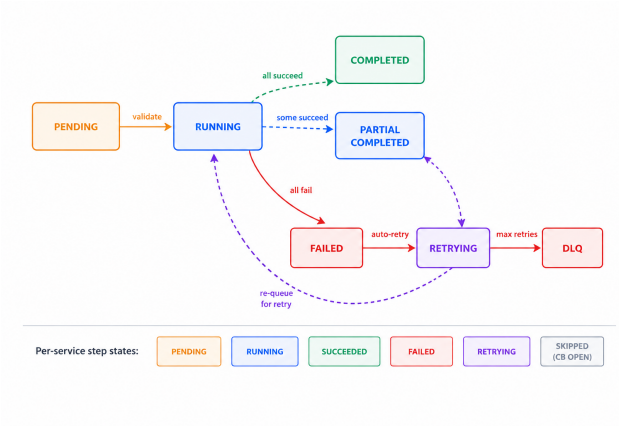

Fig. 2. Deletion request and step-state lifecycle.

C. Proof and Audit Model

Every important event is recorded as a proof event. These proof events form an append-only timeline. The system supports proof viewing, proof verification, JSON export, PDF export, and cryptographic attestation. This gives operators a way to explain what happened after the workflow finishes.

IV. Detailed Implementation

A. Backend Orchestration

The backend is written in NestJS. It coordinates request creation, step updates, proof event storage, and admin endpoints. It also exposes list endpoints so the frontend can show dashboard data, history data, and proof data.

We added support for:

- `GET /deletions`
- `GET /deletions/:id/proof`
- `GET /deletions/:id/proof/verify`
- `GET /health/all`
- `GET /admin/circuits`
- `GET /admin/sla-violations`
- `POST /deletions/bulk`

The backend also supports an SSE endpoint so the UI can receive near real-time request updates instead of waiting for manual refresh.

B. Cleanup Services

The cleaning service handles the deletion work independently. This separation makes responsibilities simple and in line with the real system of different teams having different stores. The master data service will delete the user line. The cache cleaning service clears the Redis key. The search cleaning service will delete the search documents. Backup service records backup and cleans up metadata. The analysis and cleaning service shows the delay and final consistency behavior.

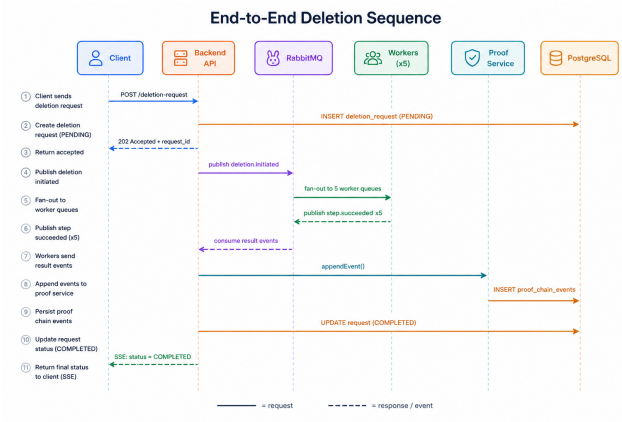

Fig. 3. Frontend pages for tracking and proof inspection (end-to-end deletion flow).

C. Frontend

The frontend is built with React and TypeScript. It includes six major views:

- Overview dashboard
- Submit Request page
- History page
- Proof and audit view
- Demo Users page
- Bulk Upload page
- Admin page

The Overview page shows the request count and selected request details. The proof view shows a readable event timeline, not the original JSON. The batch upload page accepts CSV files and returns the results of each line. The management page shows the service health status, circuit breaker status, SLA violations, and links to Grafana, RabbitMQ UI and Jaeger UI.

D. Proof Verification and Attestation

Proof integrity is supported through a tamper-evident hash chain. In addition, the system includes a cryptographic attestation flow using Ed25519 signatures. This makes the demo stronger because the system does not only say that deletion happened; it can also sign the attested result for later verification.

V. Reliability, Failure Handling, Consistency, and Security

One of the most important parts of the project is the retry design. In our system, cache cleanup uses RabbitMQ retry queues with delays of **5 seconds, 10 seconds, and 20 seconds** before finally moving the message to a dead letter queue. On each temporary failure, the service publishes a `DeletionStepRetrying` event and reschedules the original work. After three failed retries, the message goes to the DLQ and the step becomes failed.

Fig. 4. Retry and dead letter queue workflow in EraseGraph.

Fig. 5. Idempotency guard for duplicate event protection.

A. Duplicate Event Protection

Distributed systems often produce duplicates. To avoid processing the same event multiple times, we added duplicate event protection using a PostgreSQL `processed_events` table. The cache cleanup service inserts the event ID before running cleanup. If the insert conflicts, the event is treated as a duplicate, acknowledged safely, and a proof event is recorded.

B. Circuit Breaker

We added a simple but useful circuit breaker design stored in Redis. After **three consecutive failures**, the service state becomes *OPEN*. It stays open for **30 seconds**, during which new first-attempt messages are skipped. After the cooldown period, the state becomes *HALF_OPEN* and allows one test request. If that test succeeds, the circuit closes again. If not, it reopens.

This helps show failure isolation. One bad service should not make the rest of the system impossible to observe or reason about.

C. Consistency Tradeoffs

EraseGraph does not pretend all deletion targets update at the same speed. The coordinator state is strongly tracked from the API point of view, but downstream cleanup is eventually consistent. This is most visible in analytics cleanup, which is intentionally delayed. As a result, a request can temporarily appear as *PARTIAL_COMPLETED* even though the system is still healthy. We consider this honest system behavior, not a bug.

D. Security and Compliance Considerations

Because this system deals with deletion evidence, integrity matters. We added proof verification and Ed25519-based attestation to make the proof chain more trustworthy. We also used a simple service-token check in the optional API gateway

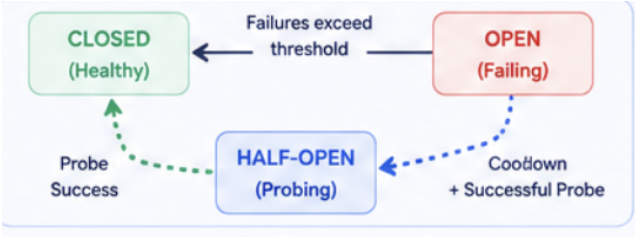

Fig. 6. Circuit breaker state transitions in EraseGraph.

for internal API protection. It shows the right direction for auditability and controlled service access.

VI. Deployment and DevOps

We supported two deployment modes.

A. Local Deployment

For local development, we used Docker Compose. This mode is useful for fast testing and demo setup. It includes the frontend, backend, cleanup services, RabbitMQ, Redis, PostgreSQL, Jaeger, Prometheus, and Grafana.

B. Cloud Deployment

For cloud deployment, we used Google Kubernetes Engine. This part of the project required extra work because our team was building on Apple Silicon while GKE nodes run on `amd64`. We had to rebuild service images using `docker buildx --platform linux/amd64` and push them to Docker Hub before rollout. We also fixed several cloud-specific issues, such as PostgreSQL initialization on persistent volumes, frontend API base URL problems, and service exposure for public demo access.

The final cloud environment provides a live frontend and public monitoring endpoints. This was important because it showed that the system worked beyond a local-only demo.

VII. Testing and Evaluation

We used a mix of unit tests, frontend tests, integration-style validation, and scenario-based demos.

A. Unit and Feature Tests

The project includes tests for bulk deletion, circuit breaker behavior, DLQ replay, and duplicate event handling. For example, circuit breaker tests validate the transition from *CLOSED* to *OPEN* after three failures and from *OPEN* to *HALF_OPEN* after the cooldown window. Bulk deletion tests validate CSV parsing, blank-row skipping, duplicate-row handling, and controller input validation.

B. End-to-End Scenarios

We validated the following key scenarios:

1) Normal deletion request where all services succeed.
2) Temporary failure that retries and later succeeds.
3) Permanent failure that reaches the DLQ.
4) Circuit breaker skip behavior.
5) Delayed analytics cleanup showing eventual consistency.
6) Proof viewing and proof verification.
7) Bulk upload with created and skipped rows.

C. Evaluation Results

Our final system successfully demonstrated all core project goals. The most useful result was not raw performance, but correctness and visibility. Reviewers can see request status, step status, queue behavior, proof events, traces, and health data. This makes the system much easier to debug and explain than a hidden background workflow.

VIII. Results and Discussion

The final version of EraseGraph achieved the main goals we set at the beginning of the project.

First, it supports **distributed deletion propagation** across multiple services. Second, it supports **operator visibility** through dashboard, history, proof, and admin pages. Third, it supports **reliability mechanisms** such as retries, DLQ, duplicate event protection, and circuit breaker logic. Fourth, it supports **auditability** through proof timelines and cryptographic attestation. Fifth, it supports **real deployment** on GKE.

At the same time, there are still restrictions. Our project uses a controlled demonstration data model instead of a large enterprise identity map. We didn't conduct a large-scale high-sencurrency load test. Some safety components are educational, not productive. In addition, the public exposure of management tools requires stronger authentication in actual deployment.

Even with these limitations, we believe that this project is also powerful for distributed system courses, because it shows many difficult topics working together in one system: asynchronous messaging, status tracking, retry, observability, final consistency, auditability and cloud deployment.

IX. Team Contributions

TABLE I
Team contribution summary

Member	Main contribution
Haoyuan Shan	Final content planning, Kubernetes/cloud deployment, backend integration closure, and documentation coordination
Vritika Malhotra	Retry and DLQ workflow, idempotency, proof-related backend work, dashboard support, and bulk CSV deletion
Sakshat Patil	Frontend real-time UX, history/proof/admin views, and SLA monitoring support
Asim Mohammed	Consistency semantics, backend/admin workflow support, and status behavior refinement

This project required close integration across all team members. Frontend pages depended on backend endpoints, admin views depended on observability and status logic, and cloud deployment required stable service images and working environment configuration.

X. Conclusion and Future Work

In this project, we built EraseGraph, a distributed deletion platform that regards deletion as a real workflow, not a single database command. The system coordinates multi-service cleaning, tracks status conversion, records proof events, handles retry and fault isolation, and displays the results on the user-friendly front end. We also deployed the project on GKE and connected it to the observability tool for a similar production demonstration.

The main lesson of this project is that deletion in distributed systems is not just a data problem. This is also a problem of workflow, reliability, observability and auditing. By making each part visible, we have created a system that is easier to reason and interpret.

There are several practical next steps for future work:

1) Add stronger authentication and authorization for admin and observability endpoints.
2) Add larger-scale load testing and more concurrency stress scenarios.
3) Replace the demo-oriented identity model with a richer subject-resolution model across services.
4) Add more services and database contain user's information or access to see how we can delete the information or remove access in one time.

References

[1] RabbitMQ, "RabbitMQ Documentation." [Online]. Available: https://www.rabbitmq.com/docs/

[2] Kubernetes, "Kubernetes Documentation." [Online]. Available: https://kubernetes.io/docs/

[3] Grafana Labs, "Grafana Documentation." [Online]. Available: https://grafana.com/docs/

[4] Jaeger Authors, "Jaeger Documentation." [Online]. Available: https://www.jaegertracing.io/docs/2.17/